Document number:	P1661R0
Date:	2019-06-13
Audience:	Library Evolution Working Group
Reply-to:	*Tomasz Kamiński <tomaszkam at gmail dot com>*

Remove dedicated precalculated hash lookup interface

1. Introduction

This paper proposed to remove the precalculated hash lookup interface (as proposed in P0920R1: Precalculated hash values in lookup), as this functionality can be implemented by the user in less error-prone manner, via heterogeneous lookup for unordered containers (P0919R3).

[Note: The implementation of hashers from this paper are included only to illustrate how precalculated hash lookup can be implemented. They are not proposed for standardization.]

2. Revision history

2.1. Revision 0

Initial revision.

3. Motivation and Scope

Contrary to information presented in the P0920R1: Precalculated hash values in lookup, the C++20 is already providing efficient way to precalculate hash value for multiple lookups, via heterogeneous lookup for unordered containers (P0919R3).

To illustrate, the following code provides a type-safe implementation for the `std::unordered_map` with `std::string` key:

```
template<typename Key>
class PrehashedStringKey
{
public:
  using key_type = Key;

  explicit PrehashedStringKey(std::size_t h, key_type v)
   : key_(std::move(v)), hash_(h)
  {}

  std::size_t hash() const
  { return hash_; }

  key_type const& key() const&
  { return key_; }

  key_type&& key() &&
  { return std::move(key_); }

  friend bool operator==(PrehashedStringKey const& lhs, std::string_view rhs)
  { return lhs.key() == rhs; }

  template<typename U>
  friend bool operator==(PrehashedStringKey const& lhs, PrehashedStringKey<U> const& rhs)
  { return lhs.key() == rhs.key(); }

private:
  key_type key_;
  std::size_t hash_;
};

struct PrecalculationEnabledStringHasher
{
   using transparent_key_equal = std::equal_to<>;

   template<typename T>
   using prehashed_key_type = PrehashedStringKey<T>;

   std::size_t operator()(std::string_view sv) const
   { return std::hash<std::string_view>{}(sv); }
```

```
    template<typename T>
    std::size_t operator()(prehashed_key_type<T> const& v) const
    { return v.hash(); }

    template<typename T>
      requires std::is_convertible_v<T, std::string_view>
    prehashed_key_type<T> precompute(T t) const
    {
      std::size_t hash = operator()(std::string_view(t));
      return prehashed_key_type<T>(hash, std::move(t));
    }
};
```

The above interface introduces a `precompute` function that bundles given key value (regardless if it is `std::string`, `std::string_view`, or `char const*`), that can be used with multiple `unordered_map` that uses the same hasher. To illustrate the example the from Motivation section of P0920R1, will now look as follows:

```
std::array<std::unordered_map<std::string, int, PrecalculationEnabledStringHasher>, array_size> maps;

void update(const std::string& user)
{
  const auto hashedUser = maps.front().hash_function().precompute(user);
  for(auto& m : maps) {
    auto it = m.find(hashedUser);
    // ...
  }
}
```

3.1. Eliminate the possibility of pairing errors

The obvious advantage of the above solution is that it eliminates the possibility of pairing key with incorrect hash value when passed to `find` function. For example, the following code contains undefined behaviour, due paring of `user2` with `hash1`:

```
std::array<std::unordered_map<std::string, int>, array_size> maps;

void update(const std::string& user1, const std::string& user2)
{
  const auto hash1 = maps.front().hash_function()(user1);
  const auto hash1 = maps.front().hash_function()(user2);
  for(auto& m : maps) {
    auto it1 = m.find(user1, hash1);
    auto it2 = m.find(user2, hash1); // undefined behaviour
    // ...
  }
}
```

However, such kind of error is not possible in case of `PrehashedStringKey` as we are bundling the key value with the appropriate hash:

```
std::array<std::unordered_map<std::string, int, PrecalculationEnabledStringHasher>, array_size> maps;

void update(const std::string& user)
{
  const auto hashedUser1 = maps.front().hash_function().precompute(user1);
  const auto hashedUser2 = maps.front().hash_function().precompute(user2);
  for(auto& m : maps) {
    auto it1 = m.find(hashedUser1);
    auto it2 = m.find(hashedUser2);
    // ...
  }
}
```

The difference in the above interface can be compared with the difference between using a range, versus passing iterator/sentinel as separate arguments.

3.2. Providing type-safety for key

Similarly to above, the currently proposed interface does not provide type-safety for key — it is possible to accidentally pass a hash of unrelated type to the `find` function, and thus causing undefined behaviour.

To illustrate, lets consider following example of correct code:

```
std::unordered_map<std::chrono::milliseconds, int> m1;
std::unordered_map<std::chrono::milliseconds, int> m2;

void update(std::chrono::seconds secs)
```

```
{
   const auto hash = m1.hash_function()(secs);
   {
      auto it1 = m1.find(secs, hash);
      // ...
   }
   {
      auto it2 = m2.find(secs, hash);
      // ...
   }
}
```

This code will silently break (undefined behaviour) in situation when one of the maps would be changed to use `std::chrono::seconds` as key:

```
std::unordered_map<std::chrono::milliseconds, int>> m1;
std::unordered_map<std::chrono::seconds, int>> m2;

void update(std::chrono::seconds secs)
{
   vconst auto hash = m1.hash_function()(secs);
   {
      auto it1 = m1.find(secs, hash);
      // ...
   }
   {
      auto it2 = m2.find(secs, hash); // undefined behaviour
      // ...
   }
}
```

[Note: That above code still compiles, as `std::chrono::seconds` are convertible to keys of both `m1` (`std::chrono::milliseconds`) and `m2` (`std::chrono::seconds`).]

However, in case if wrapper similar to `PrecomptedStringHash` would be used (see Implementability section), such mistakes would be reported as compilation error:

```
std::unordered_map<std::chrono::milliseconds, int, PrecalculationEnabledHasherFor<std::chrono::milliseconds>> m1;
std::unordered_map<std::chrono::seconds, int, PrecalculationEnabledHasherFor<std::chrono::seconds>> m2;

void update(std::chrono::seconds secs)
{
   const auto hashed = m1.hash_function().precompute(secs);
   {
      auto it1 = m1.find(hashed);
      // ...
   }
   {
      auto it2 = m2.find(hashed); // ill-formed
      // ...
   }
}
```

3.3. Mixing different hashers

Another drawback of current interface, is that it allows silently mixing of `unordered_map` instances, that use different hashers. This applies both for the type of the hasher and its state.

To illustrate, following code contains undefined behavior, due hash mismatch:

```
std::unordered_map<std::string, int, MurmurHash> m1;
std::unordered_map<std::string, int, CityHash> m2;

void update(std::string const& secs)
{
   const auto hash = m1.hash_function()(secs);
   {
      auto it1 = m1.find(secs, hash);
      // ...
   }
   {
      auto it2 = m2.find(secs, hash); // undefined behavior
      // ...
   }
}
```

Above problem, can be immediately addressed, by tagging the key/hash pair produced by the hasher with the hash type (see Implementability section).

```
std::unordered_map<std::string, int, PrecalculationEnabledHasher<MurmurHash>> m1;
std::unordered_map<std::string, int, PrecalculationEnabledHasher<CityHash>> m2;

void update(std::string const& secs)
{
   const auto hashedKey = m1.hash_function()(secs);
   {
      auto it1 = m1.find(hashedKey);
      // ...
   }
   {
      auto it2 = m2.find(hashedKey); // ill-formed
      // ...
   }
}
```

However, all of above does not address the possibility of using hasher of the same type, but having different state. [Note: This is not an issue is all hasher use the same program-wide state.]

One of the possible options is to introduce a check to the `find` method, that will rehash the object and compare it with the provided hash value. However, such solution eliminates are performance gains of this feature, making it more suited for the debug builds.

Use of the `PrehashedKey` approach, provides alternative solutions. Firstly, we can store the state of the hasher (salt) in key/hash bundle, and validating it inside `operator()`:

```
template<typename T>
std::size_t operator()(prehashed_key_type<T> const& v) const
{
  if (v.salt() != hash_.salt())
    report_error();
  return v.hash();
}
```

Above solution is reducing performance impact (comparing state of hasher is usually less expensive than rehashing of a key), and can be eliminated in case of stateless hasher object.

Secondly, we can mitigate hasher state mismatch errors, by using different hasher (and `PrehashedKey`) for each container definition. This can be achieved by adding an otherwise unused tag template parameter to `PrecalculationEnabledHasher` and `PrehashedKey`.

3.4. Summary

Examples presented above show that the "power user" that need to squeeze the extra performance from lookup of multiple `unordered_map` with the same hashes (type and state) are already able to achieve that goal via custom hasher.

In addition, with this customized implementation, each user can take a different approach regarding the unavoidable safety versus speed trade-off for the solution, e.g.:

- always compare the state of hasher in `precompute` method,
- store reference to key instead of copy in `prehashed_key`,
- use different hasher for each container definition (via tagging).

making the solution more fitting to the specific use case.

Finally, the hasher based solution makes it possible to localize the usage error-prone interface to only specific part of the code. This is not possible if the lookup mechanism is included as part of the `unordered_map` interface.

In light of the above author believes that dedicated precalculated lookup interface shall be removed from the standard, in favor of custom user-provided implementation, that suits their need.

4. Proposed Wording

Revert the changes introduced in by P0920R1.

5. Implementability

Below you may find the general implementation of the hash wrapper, that allow to extend any existing hasher with precalculated hash lookup functionality.

```
template<typename Key, typename Hasher>
struct PrehashedKey
{
  using key_type = Key;

  template<typename U>
  explicit PrehashedKey(std::size_t h, U&& u)
   : key_(std::forward<U>(u)), hash_(h)
  {}

  std::size_t hash() const
  { return hash_; }

  key_type const& key() const&
  { return key_; }

  key_type&& key() &&
  { return std::move(key_); }

private:
  key_type key_;
  std::size_t hash_;
};

template<typename Equal, typename Hasher>
class PrecalculationEnabledHashEqual
{
  [[no_unique_address]] Equal equal;

public:
  using is_transparent = void;

  template<typename T>
  using prehashed_key_type = PrehashedKey<T, Hasher>;

  PrecalculationEnabledHashEqual() = default;
  PrecalculationEnabledHashEqual(Equal e) : equal(std::move(e)) {}

  template<typename T, typename U>
    requires std::is_invocable_v<Equal const&, T const&, U const&>
  decltype(auto) operator()(T const& lhs, U const& rhs) const
  {
     return equal(lhs, rhs);
  }

  template<typename T, typename U>
    requires std::is_invocable_v<Equal const&, T const&, U const&>
  decltype(auto) operator()(prehashed_key_type<T> const& lhs, U const& rhs) const
  {
     return equal(lhs.key(), rhs);
  }

  template<typename T, typename U>
    requires std::is_invocable_v<Equal const&, T const&, U const&>
  decltype(auto) operator()(T const& lhs, prehashed_key_type<U> const& rhs) const
  {
     return equal(lhs, rhs.key());
  }

  template<typename T, typename U>
    requires std::is_invocable_v<Equal const&, T const&, U const&>
  decltype(auto) operator()(prehashed_key_type<T> const& lhs, prehashed_key_type<U> const& rhs) const
  {
     return equal(lhs.key(), rhs.key());
  }
};

template<typename Hasher>
struct HasherEquailty
{ using type = std::equal_to<>; };

template<typename Hasher>
  requires (requires () { typename Hasher::transparent_key_equal; })
struct HasherEquailty<Hasher>
{ using type = typename Hasher::transparent_key_equal; };

template<typename Hasher>
class PrecalculationEnabledHasher
{
  [[no_unique_address]] Hasher hasher;
```

```
public:
   using transparent_key_equal = PrecalculationEnabledHashEqual<HasherEquailty<Hasher>, Hasher>;

   template<typename T>
   using prehashed_key_type = PrehashedKey<T, Hasher>;

   PrecalculationEnabledHasher() = default;
   PrecalculationEnabledHasher(Hasher h) : hasher(std::move(h)) {}

   template<typename T>
     requires std::is_invocable_r_v<std::size_t, Hasher const&, T const&>
   std::size_t operator()(T const& t) const
   { return hasher(t); }

   template<typename T>
   std::size_t operator()(prehashed_key_type<T> const& v) const
   { return v.hash(); }

   template<typename T>
     requires std::is_invocable_r_v<std::size_t, Hasher const&, std::decay_t<T> const&>
   prehashed_key_type<std::decay_t<T>> precompute(T&& t) const
   {
     std::size_t hash = operator()(std::as_const(t));
     return prehashed_key_type<std::decay_t<T>>(hash, std::forward<T>(t));
   }
};

template<typename T>
using PrecalculationEnabledHasherFor = PrecalculationEnabledHasher<std::hash<T>>;
```

With above code, the `PrecalculationEnabledStringHasher` is equivalent to `PrecalculationEnabledHasherFor<std::string_view>`.

6. Acknowledgements

Special thanks and recognition goes to Sabre (http://www.sabre.com) for supporting the production of this proposal and author's participation in standardization committee.

7. References

1. Mateusz Pusz, "Precalculated hash values in lookup" (P0920R1, http://wg21.link/p0920r1)
2. Mateusz Pusz, "Heterogeneous lookup for unordered containers" (P0919R3, http://wg21.link/p0919r3)
3. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4762, https://wg21.link/n4762)